🤖 LLM Interview Prep Guide — Hinglish Edition

For AI Engineers with 5.5 YOE | Complete Fundamentals + Scenario Questions

---

📚 SECTION 1: LLM FUNDAMENTALS

Q1. What are Foundation Models, and how have they changed AI engineering?

Easy Explanation: Foundation model ek bahut bada, pre-trained model hota hai jo massive data pe train kiya gaya hota hai. Ise ek "base" ki tarah samjho — ek baar train karo, phir hazaar alag-alag tasks ke liye use karo (fine-tune karke ya as-is).

Pehle kya tha: Har task ke liye alag model banana padta tha. Sentiment analysis ke liye alag, translation ke liye alag.

Ab kya hai: GPT-4, BERT, LLaMA jaise foundation models ek baar train hote hain aur phir fine-tune hokar medical, legal, coding — sab mein kaam karte hain.

Interview Answer:

> "Foundation models are large pre-trained neural networks trained on vast, diverse datasets — text, code, images — using self-supervised learning. They've fundamentally changed AI engineering by enabling **transfer learning at scale**: instead of building task-specific models from scratch, we now fine-tune or prompt a single foundation model. This reduced time-to-production dramatically, democratized AI (via APIs), and shifted the engineer's job from model training to **prompt engineering, fine-tuning, RAG pipelines, and evaluation**. Examples: GPT-4 (OpenAI), Claude (Anthropic), Gemini (Google), LLaMA (Meta)."

Q2. What is a Large Language Model (LLM), and how does it work?

Easy Explanation: LLM ek aisa model hai jo text ko predict karta hai — "next word kya aayega?" yahi uski core job hai. Jab tune "How are" likha, to model "you" predict karta hai. Yahi bar bar karte karte pura sentence banta hai.

Training mein model internet ka bahut saara text padh chuka hota hai aur patterns seekh leta hai — grammar, facts, reasoning sab.

Interview Answer:

> "An LLM is a deep learning model, typically based on the **Transformer architecture**, trained on massive text corpora using a next-token prediction objective (autoregressive) or masked token prediction. During training, it learns statistical patterns, world knowledge, and reasoning from billions of parameters. At inference, given a prompt, it generates tokens one-

> by-one using learned probability distributions over its vocabulary. Models like GPT-4 have 100B+ parameters. The 'large' refers to both model size and training data scale."

Q3. Inside ChatGPT: What Happens After You Hit Enter?

Easy Explanation: Step by step samjho — jab tu Enter dabata hai:

1. **Tokenization** — tera text words/subwords mein toot jaata hai
2. **Embedding** — har token ek vector banta hai
3. **Transformer layers** — attention aur FFN layers se pass hota hai
4. **Logits** — model har possible next token ka score deta hai
5. **Sampling** — temperature/top-p se ek token choose hota hai
6. **Repeat** — jab tak `<EOS>` token na aaye

Interview Answer:

> "When you hit Enter: (1) Input is **tokenized** into subword tokens. (2) Each token is converted to an **embedding vector**. (3) **Positional encodings** are added. (4) The sequence passes through **N Transformer decoder layers** — each with masked self-attention, cross-attention (if applicable), and feed-forward layers. (5) The final layer projects to **logits** over the vocabulary (~50K tokens). (6) A **sampling strategy** (greedy, temperature, top-p) selects the next token. (7) That token is appended and the process repeats **autoregressively** until a stop token or max length is hit. In ChatGPT's case, there's also an RLHF-tuned layer for alignment and a safety classifier."

Q4. What is the Transformer Architecture and how does it work?

Easy Explanation: Transformer ek neural network architecture hai jo 2017 mein "Attention is All You Need" paper mein aaya. Pehle RNNs hote the jo sequence ko ek ek step process karte the — slow aur long-range dependencies miss karte the. Transformer ne sab kuch parallel kar diya "attention" mechanism ke through.

Interview Answer:

> "The Transformer is a neural architecture based entirely on **attention mechanisms**, eliminating recurrence. It has two main blocks: an **Encoder** (reads input, builds contextual representations) and a **Decoder** (generates output tokens). Each block has: (1) **Multi-Head Self-Attention** — allows each token to attend to all others, (2) **Feed-Forward Network** — position-wise MLP, (3) **Layer Normalization** and **Residual Connections**. It processes all tokens in parallel (unlike RNNs), enabling massive GPU utilization and scaling. This architecture underpins GPT (decoder-only), BERT (encoder-only), and T5 (encoder-decoder)."

Q5. What are the Key Components of the Transformer Architecture?

Easy Explanation: Transformer ke andar ye main parts hote hain — ek restaurant ki tarah socho:

- **Input Embedding** = Menu items ko numbers mein convert karna
- **Positional Encoding** = Table number dena (order matter karta hai)
- **Multi-Head Attention** = Har customer dekh raha hai kisko order dena hai
- **Feed-Forward Layer** = Kitchen processing
- **Layer Norm + Residuals** = Quality check + backup path
- **Output Layer** = Final dish serve karna

Interview Answer:

> "Key components: (1) **Token Embeddings** — map tokens to dense vectors. (2) **Positional Encoding** — injects sequence order information. (3) **Multi-Head Self-Attention** — each head learns different relationship patterns. (4) **Feed-Forward Network (FFN)** — two linear layers with GELU/ReLU activation. (5) **Layer Normalization** — stabilizes training (Pre-LN vs Post-LN). (6) **Residual Connections** — skip connections prevent vanishing gradients. (7) **Softmax Output Head** — projects to vocabulary distribution. In decoder-only models (GPT), we also have **Causal Masking** to prevent attending to future tokens."

Q6. What is Tokenization in LLMs?

Easy Explanation: Tokenization matlab text ko chhote pieces (tokens) mein todna. Model directly text nahi samajhta — numbers samajhta hai. Isliye pehle text → tokens → token IDs.

"unhappiness" → ["un", "happiness"] ya ["un", "hap", "pi", "ness"] depending on the tokenizer.

Interview Answer:

> "Tokenization is the process of converting raw text into a sequence of **tokens** (subword units), which are then mapped to integer IDs that the model processes. It's not word-splitting — modern tokenizers use subword algorithms like **BPE, WordPiece, or SentencePiece** to balance vocabulary size and coverage. A typical vocabulary is 32K–100K tokens. Tokenization affects model performance significantly: rare words get split into multiple tokens (increasing sequence length), and domain-specific terms may be poorly tokenized. The tokenizer is tied to the model — you must use the same tokenizer at inference as during training."

Q7. Explain BPE (Byte Pair Encoding)

Easy Explanation: BPE ek compression algorithm hai. Shuruaat mein har character ek token hoti hai. Phir jo pairs sabse zyada saath aate hain unhe merge karte hain. Jab tak vocabulary size reach na ho jaaye.

Example: "l o w e r" → "lo wer" → "lower" (agar ye frequently aata hai)

Interview Answer:

> "BPE starts with a character-level vocabulary and **iteratively merges the most frequent adjacent token pairs** until the desired vocabulary size is reached. For example: corpus has 'l-o-w' appearing frequently → merge to 'low'. This creates a vocabulary of common subwords. GPT-2/GPT-4 use **Byte-level BPE** — starting from bytes (256), not characters, making it language-agnostic and handling any Unicode text. Advantages: compact vocabulary, handles OOV words by decomposing them, good balance between word-level and character-level. Disadvantage: different text can tokenize the same word differently based on context."

Q8. Explain WordPiece and SentencePiece

Easy Explanation:

- **WordPiece** (BERT wala): BPE jaisa hi hai but merge criterion alag — likelihood maximize karta hai, frequency nahi. Subwords pe `##` prefix lagta hai (e.g., "playing" → "play", "##ing")
- **SentencePiece** (LLaMA, T5 wala): Language-agnostic hai, spaces ko bhi treat karta hai as tokens. Raw text se directly kaam karta hai bina pre-tokenization ke.

Interview Answer:

> "**WordPiece** (used by BERT, DistilBERT): Similar to BPE but merges based on **maximizing likelihood** of the training data rather than raw frequency. Continuation tokens are marked with `##`. Requires pre-tokenized (whitespace-split) input. **SentencePiece** (used by T5, LLaMA, Gemma): A **language-agnostic** tokenizer that treats the input as a raw stream of characters (including spaces as `▁`). It implements both BPE and Unigram LM algorithms. Key advantage: no language-specific pre-tokenization needed, making it ideal for multilingual models. Also fully reversible — you can reconstruct original text from tokens."

Q9. What is Positional Encoding, and Why is it Needed?

Easy Explanation: Transformer mein attention sab tokens ko ek saath dekhta hai — to usse kaise pata chalega ki "cat sat on mat" mein "cat" pehle aaya aur "mat" baad mein? Positional encoding ek "position signal" inject karta hai har token ke embedding mein.

Soch lo — ek classroom mein sab students ko seat number milna — taki teacher jaane ki answer kaun diya.

Interview Answer:

> "Transformers process all tokens in parallel — unlike RNNs, they have no inherent notion of order. Positional encoding adds **position-dependent signals** to token embeddings so the

> model can distinguish 'dog bites man' from 'man bites dog'. Original Transformer used **sinusoidal encoding** (fixed, mathematical sin/cos functions of different frequencies). Modern models use **learned positional embeddings** (GPT-2) or **relative positional encodings** like RoPE (LLaMA) or ALiBi (MPT). RoPE is now dominant because it generalizes better to sequences longer than seen during training."

Q10. What are Embeddings?

Easy Explanation: Embedding matlab kisi cheez ko numbers (vector) mein represent karna. "king" → [0.2, -0.5, 0.8, ...] (768 ya 1024 dimensions mein). Similar words ke vectors similar hote hain — "king" aur "queen" ke vectors close hote hain vector space mein.

Interview Answer:

> "Embeddings are **dense, low-dimensional vector representations** of discrete objects (tokens, sentences, documents). In LLMs, a token embedding maps each vocabulary item to a learned vector (e.g., 768-dim for BERT, 4096-dim for LLaMA-7B). They encode semantic meaning — similar concepts cluster together in embedding space (king - man + woman ≈ queen). Types: (1) **Token embeddings** — input layer, (2) **Positional embeddings** — encode position, (3) **Sentence/document embeddings** — from models like SBERT, used in RAG for similarity search. The embedding matrix is usually the largest parameter matrix in an LLM."

Q11. Explain Q, K, V in Attention

Easy Explanation: Library analogy:

- **Query (Q)** = Tu kya dhundh raha hai (search query)
- **Key (K)** = Library ke books ke labels
- **Value (V)** = Actual book ka content

Tera query har key ke saath match hota hai → similarity score milta hai → un scores se values weight hoti hain → final answer milta hai.

Interview Answer:

> "In attention, every token produces three vectors via learned projections: **Query (Q)** — what this token is looking for; **Key (K)** — what each token offers/contains; **Value (V)** — the actual information to retrieve. Attention score = $\text{softmax}(QK^T / \sqrt{d_k}) \times V$. The dot product $Q \cdot K^T$ measures compatibility between tokens. After softmax normalization, these become attention weights applied to V. This lets every token gather information from relevant tokens in the sequence. In self-attention, Q, K, V all come from the same sequence. In cross-attention (encoder-decoder), Q comes from decoder, K/V from encoder."

Q12. What is Self-Attention and How Does it Work?

Easy Explanation: Self-attention matlab — ek sentence ke andar har word dusre words ko "attend" karta hai. "The animal didn't cross the street because it was too tired" — yahan "it" kis ko refer karta hai? Self-attention model ko samjhaata hai ki "it" → "animal" ko refer kar raha hai.

Interview Answer:

> "Self-attention allows **each token in a sequence to attend to every other token** in the same sequence to build contextual representations. Process: (1) Compute Q, K, V from input X via weight matrices WQ, WK, WV. (2) Compute attention scores: $A = QK^T / \sqrt{d_k}$. (3) Apply softmax to get attention weights (sum to 1). (4) Weighted sum of V: Output = softmax(A) × V. Complexity is **$O(n^2)$** in sequence length — the main bottleneck for long contexts. Self-attention enables **global context** — every token can directly influence every other, solving the long-range dependency problem of RNNs."

Q13. Why Scale by $\sqrt{d_k}$ in Attention?

Easy Explanation: Dot product QK^T bahut bada ho jaata hai jab d_k (dimension) bada ho. Agar values bahut badi hain, toh softmax "peaky" ho jaata hai — basically ek hi token pe saara attention jaata hai, gradients vanish ho jaate hain. $\sqrt{d_k}$ se divide karke hum dot products ko reasonable range mein rakhte hain.

Interview Answer:

> "When d_k is large, dot products Q·K can grow large in magnitude because they're sums of d_k terms. Large values push softmax into regions of **extremely small gradients** (softmax saturates). Dividing by $\sqrt{d_k}$ keeps the dot products in a variance-1 regime (assuming Q, K have unit variance components), preventing softmax saturation and ensuring **stable gradient flow** during training. This is called **Scaled Dot-Product Attention** and is a critical numerical stability trick."

Q14. What is Causal Masking?

Easy Explanation: Decoder (GPT style) mein jab model text generate karta hai, to usse future tokens nahi dekhne chahiye — ye cheating hogi! Causal masking ek upper triangular matrix hai jo future positions ko -infinity se mask kar deta hai, taaki softmax ke baad unka weight 0 ho jaaye.

Interview Answer:

> "Causal masking (aka **autoregressive masking**) ensures that during training, when predicting token at position t, the model can only attend to tokens at positions 0 to t-1 — never future tokens. Implemented as an **upper triangular mask** added to attention scores (future positions get -∞, which becomes 0 after softmax). This is essential for **decoder-only models** (GPT, LLaMA) trained with next-token prediction — without it, the model could

> 'cheat' by looking ahead. Encoder models (BERT) don't use causal masking, allowing bidirectional attention."

Q15. What are Multi-Head Attention Mechanisms? Why Multiple Heads?

Easy Explanation: Ek single attention head ek tarah ka relationship seekhta hai. Multiple heads alag-alag types ke relationships seekhte hain simultaneously.

Ek sentence mein: ek head syntactic structure dekhta hai, doosra semantic similarity, teesra coreference — sab parallel mein. Phir sab concatenate hokar ek final vector banta hai.

Interview Answer:

> "Multi-Head Attention runs h attention operations in parallel (typically 8, 12, or 32 heads). Each head has its own WQ, WK, WV projection matrices, allowing it to attend to different **representation subspaces**. Outputs are concatenated and linearly projected: $\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1 \ldots \text{head}_h) \times WO$. **Why multiple heads?** A single head is forced to average over all relationships. Multiple heads allow the model to jointly capture: syntactic dependencies, semantic similarity, positional patterns, coreference, etc. — all simultaneously. Each head dimension = dmodel/h, so total compute stays the same as single-head attention of full dimension."

Q16. What is the Context Window in LLMs, and Why Does it Matter?

Easy Explanation: Context window = model kitne tokens ek saath dekh sakta hai. Agar context window 4096 tokens hai, to model sirf last 4096 tokens "yaad" rakh sakta hai. Usse zyada purani baat? Bhool gayi.

Ye matter karta hai kyunki: long documents process karne ke liye, long conversations maintain karne ke liye, RAG mein zyada context dene ke liye — sab ke liye bada context window chahiye.

Interview Answer:

> "The context window is the **maximum number of tokens** the model can process in a single forward pass — both input prompt + generated output combined. Examples: GPT-3.5 (4K), GPT-4-turbo (128K), Claude 3 (200K), Gemini 1.5 (1M). It matters because: (1) **Long documents** need large windows to avoid truncation, (2) **Multi-turn conversations** accumulate tokens quickly, (3) **RAG systems** benefit from fitting more retrieved chunks, (4) **Code assistants** need to see entire codebases. Increasing context window is expensive — attention is $O(n^2)$ in sequence length. Solutions: FlashAttention, GQA, sliding window attention (Mistral), and RoPE-based extrapolation."

Q17. What is Temperature in LLMs?

Easy Explanation: Temperature = model ki "creativity" ya "randomness" ka dial.

- **Low temperature (0.1–0.3):** Deterministic, predictable, always most likely word choose karta hai (good for factual QA, code)
- **High temperature (0.8–1.2):** Creative, random, unexpected words bhi choose karta hai (good for creative writing)
- **Temperature = 0:** Greedy decoding — always argmax

Technically: logits ko temperature se divide karte hain before softmax.

Interview Answer:

> "Temperature is a **scaling parameter applied to logits before softmax**. logits_scaled = logits / T. At T=1, original distribution is used. T<1 **sharpens** the distribution (makes model more confident, less diverse). T>1 **flattens** it (more uniform, more random output). T→0 approaches **greedy decoding**. In practice: T=0.0-0.3 for factual/code tasks, T=0.7-1.0 for creative tasks. Temperature only affects **sampling diversity** — it doesn't change the model's underlying knowledge or hallucination rate in a fundamental way."

Q18. Explain Top-p (Nucleus) Sampling and Top-k Sampling

Easy Explanation: Imagine model ke paas 50,000 tokens hain aur sab ke probabilities hain:

- **Top-k:** Sirf top 10 (ya k) tokens mein se choose karo, baaki ignore
- **Top-p:** Cumulative probability 0.9 (ya p) tak ke tokens rakho — agar 5 tokens mein 90% probability aa jaaye, sirf unhi mein se choose karo

Top-p zyada flexible hai — kabhi 3 tokens, kabhi 50 tokens depend karke distribution pe.

Interview Answer:

> "**Top-k sampling:** At each step, restrict sampling to only the k highest-probability tokens (e.g., k=50). Simple but rigid — if k=50, sometimes 50 meaningful tokens exist, sometimes 5. **Top-p (nucleus) sampling:** Sort tokens by probability, take the smallest set whose cumulative probability ≥ p (e.g., 0.9). This is **adaptive** — the nucleus can be 5 tokens or 500 tokens depending on how peaked the distribution is. Top-p is generally preferred. Both prevent sampling from very low-probability tokens (reduces gibberish). Often used together: apply top-k first, then top-p, then temperature, then sample."

Q19. What are Logits, and How are They Used in Text Generation?

Easy Explanation: Model ke last layer ke raw output numbers hain — ye logits hain. Abhi tak softmax nahi laga. Logits ko softmax mein daalte hain → probabilities milti hain. Phir temperature, top-p, top-k lagate hain → next token sample karte hain.

Interview Answer:

> "Logits are the **raw, unnormalized scores** output by the final linear layer of the LLM, one score per vocabulary token. Shape: [vocab_size] (e.g., 50,257 for GPT-2). They're converted to probabilities via softmax: p(token_i) = exp(logit_i) / Σ exp(logit_j). In generation: (1) Get logits for current position, (2) Apply temperature scaling, (3) Apply top-k/top-p filtering, (4) Sample from the resulting distribution. Logits are also used in **training** via cross-entropy loss — comparing predicted logits against ground truth token."

Q20. What are Skip Connections (Residual Connections) in Transformers?

Easy Explanation: Deep networks mein gradients vanish ho jaate hain (bahut chhote ho jaate hain) jab backprop karte hain. Residual connection ek "shortcut" deta hai — input directly output mein add ho jaata hai: output = F(x) + x. Is tarah gradient seedha flow kar sakta hai bina deep layers se guzre.

Interview Answer:

> "Residual (skip) connections add the **input of a sub-layer directly to its output**: output = LayerNorm(x + SubLayer(x)). This creates a gradient highway — during backpropagation, gradients can flow directly through the addition operation without vanishing, enabling training of very deep networks (Transformers have 12–96+ layers). They also help with **representation preservation** — if a layer doesn't need to transform the input much, it can learn near-zero weights and rely on the residual path. Without residual connections, Transformers of this depth would be practically untrainable."

Q21. Open-source vs Closed-source LLMs: When to Choose?

Easy Explanation:

- **Closed (GPT-4, Claude):** API call karo, pay karo, state-of-the-art performance, no control
- **Open (LLaMA, Mistral, Gemma):** Download karo, apne server pe chalao, customize karo, data privacy

Kab kya choose karein:

- Startup, quick prototype → Closed API
- Data privacy (healthcare, finance) → Open source
- Custom domain fine-tuning → Open source
- Best-in-class performance chahiye, budget hai → Closed

Interview Answer:

> "**Closed-source** (GPT-4, Claude, Gemini): No infrastructure overhead, best absolute performance, continuous updates, but expensive at scale, no data control, vendor lock-in, black-box. **Open-source** (LLaMA-3, Mistral, Phi-3, Gemma): Full control, data stays on-

> premise (critical for HIPAA/GDPR), fine-tunable, no per-token cost at scale, but requires MLOps expertise. **Choose closed when:** rapid prototyping, need SOTA quality, small team. **Choose open when:** data privacy is non-negotiable, need custom fine-tuning, high-volume inference (cost), or want to avoid vendor lock-in."

Q22. Encoder-only vs Decoder-only vs Encoder-Decoder

Easy Explanation:

- **Encoder-only (BERT):** Text padho, samjho → classification, NER, embeddings ke liye. Generate nahi kar sakta.
- **Decoder-only (GPT):** Text generate karo → chatbots, code gen, text completion
- **Encoder-Decoder (T5, BART):** Ek cheez lelo, doosri cheez generate karo → translation, summarization

Interview Answer:

> "**Encoder-only** (BERT, RoBERTa): Bidirectional attention, sees full context. Used for **understanding tasks** — classification, NER, semantic search, embeddings. Cannot generate text. **Decoder-only** (GPT-4, LLaMA, Claude): Causal/unidirectional attention. Trained for **text generation** via next-token prediction. Dominates modern LLMs because generation covers most tasks. **Encoder-Decoder** (T5, BART, mT5): Encoder processes input bidirectionally, decoder generates output autoregressively. Natural fit for **sequence-to-sequence tasks**: translation, summarization, Q&A with extraction. Modern trend: decoder-only models can do all these tasks with instruction tuning, making encoder-decoder less common."

Q23. What is KV Cache, and How Does it Speed Up Inference?

Easy Explanation: Autoregressive generation mein har token generate karne ke liye poori sequence process karni padti hai. Par pichle tokens ke K aur V vectors nahi badalte! KV cache mein hum inhe store karte hain aur dobara compute nahi karte.

Bina cache: 100 tokens generate karne ke liye 100 + 99 + 98... = 5050 operations
Cache ke saath: 100 + 1 + 1... = ~200 operations

Interview Answer:

> "During autoregressive generation, each new token requires computing attention over all previous tokens. The Key (K) and Value (V) matrices for past tokens **don't change** — recomputing them is wasteful. KV cache stores these matrices from previous steps and **reuses** them, only computing K, V for the new token. This reduces per-step complexity from O(n) to O(1) for past tokens. Tradeoff: **memory cost** — KV cache size = 2 × num_layers × num_heads × head_dim × seq_len × batch_size × precision_bytes. At long contexts (128K

> tokens), KV cache can be GBs. Addressed by: GQA (reduces heads), quantized KV cache, sliding window attention."

Q24. Autoregressive vs Masked Language Modeling

Easy Explanation:

- **Autoregressive (GPT style):** Left to right — predict next token. "The cat ___" → "sat"
- **Masked LM (BERT style):** Random tokens mask karo → predict karo. "The [MASK] sat on the mat" → "cat"

AR = generation ke liye perfect. MLM = understanding, embeddings ke liye perfect.

Interview Answer:

> "**Autoregressive LM** (Causal LM): Predicts the next token given all previous tokens, left-to-right. Training objective: maximize P(token_t | token_1...token_{t-1}). Naturally suited for **text generation**. Models: GPT family, LLaMA, Claude. **Masked LM** (MLM): Randomly masks 15% of tokens, model predicts masked tokens using **bidirectional context**. Enables richer contextual representations but cannot directly generate text. Models: BERT, RoBERTa. Modern trend: instruction-tuned autoregressive models have largely replaced MLM models even for understanding tasks, as they generalize better through prompting."

Q25. What is Model Distillation?

Easy Explanation: Bada powerful model (teacher) hai — par expensive hai. Chhota model (student) banate hain jo teacher ki behavior copy kare — soft labels (probability distributions) se seekhta hai, na sirf hard labels se. Result: faster, cheaper student model jo almost teacher jaisi performance deta hai.

Example: DistilBERT = BERT ka 40% chhota, 60% faster version.

Interview Answer:

> "Knowledge distillation trains a smaller **student model** to mimic a larger **teacher model**. Instead of training on hard labels (one-hot), the student is trained on the teacher's **soft probability distributions** (logits), which contain richer information (e.g., '0.8 cat, 0.15 lynx, 0.05 tiger' tells you more than just 'cat'). Loss = α × Cross-entropy(student_logits, true_labels) + (1-α) × KL-div(student_softmax, teacher_softmax). Used for: **LLM compression** (DistilGPT-2, DistilBERT), deployment on edge devices, reducing inference cost. Distillation can also transfer reasoning capability — 'speculative decoding' uses a small draft model + large verifier model."

Q26. What is Mixture of Experts (MoE)?

Easy Explanation: Normal transformer mein har token FFN layer mein se poori tarah guzarta hai. MoE mein FFN ke jagah kai "experts" hote hain (alag-alag sub-networks), aur har token sirf 1-2 experts ke through jaata hai (routing network decide karta hai). Iska matlab: bahut bade model parameters ho sakte hain par compute kam.

Mixtral 8x7B = 8 experts, par ek time par sirf 2 active — effectively 2x7B compute ke saath.

Interview Answer:

> "MoE replaces the dense FFN layer with N expert sub-networks + a **gating/router network** that routes each token to the top-k experts (usually k=2). Only activated experts compute — giving **sparse computation**. Total parameters are large (say 47B in Mixtral 8x7B), but active parameters per token are small (~13B). Benefits: (1) More capacity without proportional compute increase, (2) Different experts specialize in different domains/tasks. Challenges: **load balancing** (auxiliary loss to prevent expert collapse), communication overhead in distributed settings. Used in: Mixtral, GPT-4 (rumored), Gemini 1.5."

Q27. Dense vs Sparse Models

Easy Explanation:

- **Dense model:** Har token → saare parameters activate. GPT-4 agar 1T params ka hai to sab 1T compute karte hain. Predictable, zyada memory.
- **Sparse model (MoE):** Har token → sirf kuch experts activate. Bahut bada hone ke bawajood fast.

Interview Answer:

> "**Dense models** activate **all parameters** for every token (e.g., LLaMA-7B uses all 7B params for every token). Compute = model size × sequence length. **Sparse models** (MoE) activate only a **subset of parameters** per token via routing. A 47B-param Mixtral model uses ~13B active params per token — same inference speed as a 13B dense model but with 47B worth of knowledge. Dense models: simpler to train, consistent behavior, better per-parameter efficiency. Sparse models: higher capacity at lower compute, but harder to train (load balancing), less predictable memory access patterns."

Q28. What is Flash Attention?

Easy Explanation: Standard attention $O(n^2)$ memory use karta hai — bada context = GPU OOM. Flash Attention ne yahi problem solve ki. Yeh GPU memory hierarchy ka fayda uthata hai — SRAM (fast but small) aur HBM (slow but large) — aur attention ko "tiled" blocks mein compute karta hai. Same mathematical result, par much less HBM access.

Interview Answer:

> "FlashAttention (Dao et al., 2022) is an **IO-aware exact attention algorithm** that computes the same attention as standard attention but is **2-4x faster and uses O(n) memory** (vs $O(n^2)$). Key insight: standard attention writes the full N×N attention matrix to HBM (GPU global memory), which is the bottleneck. FlashAttention instead **tiles** the computation into blocks that fit in SRAM (fast on-chip memory), computing attention chunk-by-chunk and accumulating results — never materializing the full attention matrix. FlashAttention-2 further optimizes parallelism; FA-3 uses hardware-specific optimizations. Now standard in all major frameworks."

Q29. What is Grouped-Query Attention (GQA)?

Easy Explanation: Standard MHA: har head ke apne Q, K, V hote hain → bahut zyada KV cache memory. MQA (Multi-Query): sirf 1 K, V hote hain sab heads ke liye → fast but quality drop. GQA: middle ground — K, V groups mein share hote hain (e.g., 32 Q heads, 8 KV heads).

LLaMA-2, Mistral, Gemma sab GQA use karte hain.

Interview Answer:

> "**MHA**: h query heads, h key heads, h value heads — most expressive, most KV cache. **MQA** (Multi-Query Attention): h query heads, but only 1 shared K head and 1 shared V head. Drastically reduces KV cache but hurts quality. **GQA** (Grouped-Query Attention): h query heads grouped into g groups, each group sharing 1 K and 1 V head (g << h). e.g., 32 Q heads, 8 KV groups. GQA achieves **near-MHA quality with near-MQA memory efficiency**. KV cache size reduced by factor h/g. Now the default in modern models: LLaMA-2/3, Mistral, Gemma, Falcon."

Q30. How does Rotary Position Embedding (RoPE) Work?

Easy Explanation: Sinusoidal PE aur learned PE dono fixed hain — training se zyada long sequence mein generalize nahi karte. RoPE ek smarter approach hai: instead of adding position info, Q aur K vectors ko position ke hisaab se "rotate" karta hai in vector space. Yeh rotation relative positions capture karta hai naturally, aur long sequences tak extrapolate kar sakta hai.

Interview Answer:

> "RoPE encodes position by **rotating Q and K vectors** using position-dependent rotation matrices. For tokens at positions m and n, their dot product $Q_m \cdot K_n$ depends only on their **relative position (m-n)**, not absolute positions. This is implemented efficiently via complex number multiplication. Why preferred: (1) **Relative position** naturally captured — better for long contexts, (2) Better **length extrapolation** than sinusoidal PE — models like LLaMA with YaRN can extend context beyond training length, (3) Compatible with KV caching. Variants: YaRN, NTK-aware scaling extend effective context window further. Standard in: LLaMA, Mistral, Falcon, Qwen."

🦴 SECTION 2: SCENARIO-BASED QUESTIONS

Q31. Your LLM keeps ignoring your instructions. How do you make it follow structured output formats?

Interview Answer:

> "Multi-pronged approach: (1) **System prompt reinforcement** — explicitly state the format with examples: 'Always respond in this JSON schema: {...}'. (2) **Few-shot examples** — show 2-3 input-output pairs with the exact format. (3) **Structured output APIs** — use OpenAI's `response_format: {type: 'json_object'}` or Anthropic's tool use to enforce schema. (4) **Output parsers with retry** — use LangChain/Instructor: if parse fails, re-prompt with the error. (5) **Fine-tuning** — if the model consistently fails, fine-tune on (instruction, correct-format-output) pairs. (6) **Validation layer** — pydantic/jsonschema validation post-generation with fallback. For production: Instructor library or Outlines for grammar-constrained decoding."

Q32. Your LLM hits the context window limit on long documents. How do you handle it?

Interview Answer:

> "Several strategies: (1) **RAG (Retrieval-Augmented Generation)** — chunk the document, embed chunks, store in vector DB (Pinecone, Weaviate), retrieve top-k relevant chunks at query time. (2) **Hierarchical summarization** — recursively summarize chunks, then summarize summaries. (3) **Map-Reduce** — process each chunk independently (map), then combine results (reduce). (4) **Sliding window with overlap** — process overlapping windows to maintain continuity. (5) **Longer context models** — upgrade to Claude 3 (200K) or Gemini 1.5 (1M). (6) **Selective retrieval** — use LLM to identify relevant sections first, then process only those. In production RAG is the most cost-effective and scalable solution."

Q33. Your LLM does not admit when it doesn't know the answer. How do you make it say "I don't know"?

Interview Answer:

> "(1) **Explicit system prompt instruction**: 'If you are not confident about an answer, say I don't know rather than guessing. Never fabricate facts.' (2) **RAG with source grounding**: If answer not found in retrieved context, instruct model to say 'No relevant information found'. (3) **Confidence elicitation**: Ask model to rate its confidence: 'Rate your confidence 1-5. If below 3, say I don't know.' (4) **Fine-tuning on abstention examples**: Include training examples where correct answer is 'I don't know'. (5) **Self-consistency**: Generate N answers, if

> variance is high → uncertain. (6) **Calibration via RLHF**: Reward the model for saying 'I don't know' when it's actually wrong."

Q34. Your LLM generates responses that are too verbose. How do you control response length?

Interview Answer:

> "(1) **Explicit prompt instruction**: 'Respond in 2-3 sentences maximum' or 'Be concise'. (2) **max_tokens parameter**: Hard limit on generation length. (3) **Few-shot examples** showing desired length. (4) **Stop sequences**: Define tokens that stop generation (e.g., '\n\n' for short answers). (5) **System prompt framing**: 'You are a concise assistant. Never over-explain.' (6) **Fine-tuning** on concise response examples if this is a consistent problem. (7) **Presence/frequency penalty**: Penalize token repetition and long outputs (OpenAI API). (8) **Post-processing**: Summarize LLM output with another (smaller) model call."

Q35. Your LLM memorized proprietary training data and leaks it in responses. How do you prevent this?

Interview Answer:

> "(1) **Differential Privacy** during training: Add calibrated noise to gradients so individual training examples can't be extracted. (2) **Data deduplication**: Deduplicate training data — memorization correlates with repetition. (3) **System prompt guardrails**: 'Never reproduce copyrighted text, PII, or proprietary information'. (4) **Output filters**: Post-generation PII detection (regex, NER models like spaCy/Presidio). (5) **RLHF with privacy reward**: Train reward model to penalize sensitive data leakage. (6) **Canary tokens**: Plant fake sensitive strings in training data to detect memorization. (7) **Membership inference testing**: Periodically test if model can reproduce known training samples."

Q36. Your LLM coding assistant generates outdated code using deprecated libraries. How do you fix it?

Interview Answer:

> "(1) **RAG with up-to-date docs**: Index latest library documentation (e.g., crawl official docs), retrieve relevant sections at query time. (2) **System prompt with version context**: 'Use Python 3.11, pandas 2.0, use new API `df.groupby()` not `df.groupby().agg()`'. (3) **Fine-tuning on current code**: Fine-tune on recent GitHub repos with explicit version tags. (4) **Tool use / Code execution**: Let model execute code in a sandbox — if deprecated API throws error, retry with correction. (5) **Version-aware retrieval**: Filter vector DB by doc publication date. (6) **Deprecation warning injection**: Use static analysis tools to detect deprecated patterns post-generation."

Q37. Your tokenizer splits important domain terms into meaningless subword pieces. How do you fix it?

Interview Answer:

> "(1) **Add domain terms to vocabulary**: Most tokenizers support adding special tokens — `tokenizer.add_tokens(['BRCA1', 'interleukin-6', 'solidity'])`. After adding, resize model embeddings. (2) **Domain-adaptive pre-training**: Continue pre-training on domain corpus — tokenizer learns common domain patterns in BPE merges. (3) **Custom tokenizer training**: Train a new SentencePiece/BPE tokenizer on domain data, then initialize a model from scratch or use it with an existing model. (4) **Prompt-side workaround**: Add explicit instructions like 'BRCA1 is a single medical term'. (5) **Evaluate impact first**: Not all splitting degrades performance — test before optimizing."

Q38. Your Transformer's KV cache grows too large during long sequence generation. How do you manage memory?

Interview Answer:

> "(1) **Grouped-Query Attention (GQA)**: Reduce number of KV heads from H to G (e.g., 32→8), reducing KV cache by 4x. (2) **KV cache quantization**: Quantize K, V tensors to INT8/FP8 — 2-4x memory reduction with minimal quality loss. (3) **Sliding window attention**: Only maintain KV cache for last W tokens (Mistral uses W=4096). (4) **Paged KV cache (vLLM)**: Manage KV cache like OS virtual memory — dynamic allocation in non-contiguous blocks. (5) **Token eviction/pruning**: Drop low-attention-weight KV entries (H2O algorithm). (6) **FlashAttention**: Recompute attention in chunks without storing full cache. (7) **Limit batch size and max sequence length** based on GPU memory budget."

Q39. Your Transformer runs out of memory on long documents due to quadratic self-attention. How do you scale it?

Interview Answer:

> "(1) **FlashAttention**: IO-aware tiled computation — same result, $O(n)$ memory instead of $O(n^2)$. (2) **Sparse attention**: Attend only to local + global tokens (Longformer, BigBird). (3) **Linear attention**: Replace softmax with kernel approximation — $O(n)$ complexity (Performer, cosFormer). (4) **Sliding window**: Each token only attends to W neighbors (Mistral's SWA). (5) **Hierarchical processing**: Process chunks, then cross-chunk attention. (6) **Retrieval augmentation**: Don't put entire doc in context — retrieve relevant parts. (7) **Gradient checkpointing**: Trade compute for memory during training. (8) **Model parallelism**: Sequence parallelism splits attention computation across GPUs."

Q40. Your distilled student model fails on complex reasoning that the teacher model handled. How do you close the gap?

Interview Answer:

> "(1) **Chain-of-thought distillation**: Distill teacher's **reasoning traces** (CoT outputs), not just final answers. (2) **Task-specific distillation**: Focus distillation on reasoning datasets (GSM8K, MATH, ARC) rather than generic text. (3) **Intermediate layer distillation**: Match not just output logits but intermediate hidden states between teacher and student. (4) **Capacity increase**: Increase student model size — there may be a capability cliff below certain parameters. (5) **Data augmentation**: Synthesize harder examples using teacher, train student on them. (6) **Speculative decoding**: Use student as draft model + teacher as verifier — student drafts, teacher corrects. (7) **Self-play / iterative refinement**: Train student to critique and revise its own reasoning."

Q41. After RLHF alignment, your LLM became safer but lost capability on hard tasks. How do you manage the alignment tax?

Interview Answer:

> "The alignment tax is real — over-optimization on safety can hurt capability. Solutions: (1) **Better reward model**: More nuanced RM that rewards both helpfulness AND safety, not just refusals. (2) **Constitutional AI (CAI)**: Use AI feedback (RLAIF) with more nuanced principles rather than binary human preferences. (3) **KL divergence constraint**: During RLHF, constrain how far policy drifts from SFT model via KL penalty. (4) **Multi-objective RLHF**: Separate reward heads for safety and capability, tune the Pareto frontier. (5) **RLHF on capability-specific data**: Ensure RLHF training data includes hard reasoning tasks. (6) **DPO (Direct Preference Optimization)**: More stable than PPO, less prone to over-optimization. (7) **Capability evals during alignment**: Monitor capability benchmarks throughout RLHF — early stop if degradation detected."

Q42. Your RLHF-trained LLM is gaming the reward model instead of being genuinely helpful. How do you fix reward hacking?

Interview Answer:

> "Reward hacking happens when the policy exploits reward model weaknesses. Fixes: (1) **KL divergence penalty**: Penalize the policy for deviating too far from SFT model — limits exploration of reward model exploits. (2) **Ensemble reward models**: Train multiple RMs on different data splits — policy must fool all of them simultaneously (much harder). (3) **Iterative RM training**: Periodically generate responses with current policy, have humans re-label, retrain RM. (4) **Constitutional AI / RLAIF**: More scalable diverse feedback reduces blind spots. (5) **Adversarial RM evaluation**: Red-team your RM — find inputs where RM gives high score for bad outputs, fix those. (6) **PPO clip**: Constrain policy update magnitude per step. (7) **Process reward models**: Reward each reasoning step, not just final answer."

Q43. Your chatbot loses context after 10 turns in a conversation. How do you maintain long conversation context?

Interview Answer:

> "(1) **Sliding window with summary**: Keep last N turns verbatim + a rolling summary of older turns. (2) **Compression/summarization**: Periodically compress older conversation with a separate summarization call. (3) **Memory modules**: LangChain's ConversationSummaryMemory, ConversationBufferWindowMemory. (4) **External memory store**: Store key facts from conversation in a vector DB — retrieve relevant memories at each turn. (5) **Longer context model**: Upgrade to models with 128K+ context. (6) **Structured memory**: Extract entities and facts into a structured JSON 'memory state' that's included in each prompt. (7) **MemGPT / agent memory**: Use an agent that manages its own context window — decides what to keep, summarize, or retrieve."

Q44. Your chatbot fails when users switch topics mid-conversation. How do you handle topic switches?

Interview Answer:

> "(1) **Topic detection**: Classify each user turn — if topic changes significantly (cosine similarity drop between turn embeddings), trigger topic switch handling. (2) **Context reset with summary**: On topic switch, save current context to memory, start fresh context with brief background. (3) **Multi-session memory**: Maintain per-topic context windows, switch based on current topic. (4) **Explicit acknowledgment prompt**: Instruct model: 'If user switches topics, acknowledge the switch and address the new topic without assuming context from the previous topic.' (5) **Intent classification layer**: Before LLM call, classify intent/topic — route to topic-specific system prompts. (6) **State machine approach**: Explicitly manage conversation state, track current topic, handle transitions."

Q45. Your QA system always generates an answer even when no answer exists. How do you detect unanswerable questions?

Interview Answer:

> "(1) **Explicit system prompt**: 'If the answer is not in the provided context, respond with: The answer is not available in the provided context. Do not make up an answer.' (2) **Confidence threshold**: Ask model to output confidence with answer — filter below threshold. (3) **Extractive grounding check**: After generation, verify every claim appears in source context (NLI/entailment model). (4) **Separate answerability classifier**: Fine-tune a small BERT model to classify if (question, context) pair is answerable. (5) **Self-consistency**: Generate answer N times — if inconsistent, likely hallucination. (6) **RAGAS evaluation**: Use RAGAS metrics like faithfulness score to detect ungrounded answers. (7) **Instruction fine-tuning**:

> Include abstention examples in fine-tuning data — (unanswerable_Q, context) → 'I don't know'."

Q46. Your summarization system hallucinated facts not in the original article. How do you fix it?

Interview Answer:

> "(1) **Faithfulness-focused prompting**: 'Summarize ONLY using information in the provided text. Do not add any external knowledge.' (2) **Extractive pre-step**: First extract key sentences (extractive), then abstractively rephrase only those — reduces hallucination surface. (3) **NLI-based faithfulness filter**: After generation, use an NLI model (e.g., FactScore) to check each claim in summary against source — regenerate if unfaithful. (4) **FactScore / SummaC evaluation**: Integrate faithfulness metrics in pipeline, reject low-score summaries. (5) **Grounded generation**: Force model to quote source text, then compress quotes — harder to hallucinate. (6) **Fine-tuning on faithful summaries**: Use CNN/DailyMail with faithfulness annotations. (7) **Temperature reduction**: Lower temperature = less creative = less hallucination."

Q47. Your text generation repeats phrases in long outputs. How do you fix repetition?

Interview Answer:

> "(1) **Repetition penalty** (Hugging Face `repetition_penalty` > 1.0): Penalizes tokens that have already appeared — reduces exact repetition. (2) **No-repeat n-gram size**: Block any n-gram from appearing twice (`no_repeat_ngram_size=3`). (3) **Frequency penalty** (OpenAI API): Penalizes tokens proportional to how often they've appeared. (4) **Presence penalty** (OpenAI API): Flat penalty for any token that has appeared at all. (5) **Diverse beam search**: Use beam groups that are encouraged to be different from each other. (6) **Nucleus/top-p sampling**: Greedy decoding often leads to loops — sampling breaks them. (7) **Prompt instruction**: 'Do not repeat phrases. Vary your language.' (8) **Stop if repetition detected**: Post-process: detect n-gram repetition, truncate and regenerate from that point."

🎯 QUICK REVISION CHEATSHEET

Topic	Key Points
Transformer	Attention + FFN + LN + Residuals
Attention	$Q \cdot K^T / \sqrt{d_k}$ → softmax → ×V
Tokenization	BPE (frequency merge), WordPiece (likelihood), SentencePiece (raw text)

Topic	Key Points
Context Window	Max tokens in one pass, $O(n^2)$ attention bottleneck
Temperature	Low=deterministic, High=creative
Top-p vs Top-k	Top-p adaptive nucleus, Top-k fixed
KV Cache	Store K,V for past tokens, reuse at each step
GQA	Share KV heads in groups, reduce cache
FlashAttention	Tiled IO-aware attention, $O(n)$ memory
RoPE	Rotate Q,K by position → relative PE, better extrapolation
MoE	N experts, route each token to k=2, sparse compute
Distillation	Student mimics teacher's soft logits
RAG	Retrieve relevant chunks → augment context
RLHF	Human preferences → reward model → PPO/DPO

All the best for your interview! You've got this! 🚀